

SOFTWARE ENGINEERING – LEVEL 3 MINI-PROJECT

DELIVERABLE 1 - SYNOPSIS / PROJECT PROPOSAL

Simple Shell — POSIX-Compliant Command Line Interpreter in C

1.1 Project Title and Group

Field	Details
Team Members	Anshul Poovaiah K (PES1UG24CS071) Archana Shivakumar (PES1UG24CS078) Ashlesh R V (PES1UG24CS088) Atishay Jain (PES1UG24CS097)
Project Title	Simple Shell — POSIX-Compliant Command Line Interpreter in C
Group/Team Number	Team 10 (G-10)
Project ID	SH-2026-10
Course	UE24CS341A - Software Engineering, V Semester
Development Methodology	Agile (Scrum, 6 sprints)
Technology Stack	C (POSIX System Calls, GCC, GNU Make)

Field	Details
Complexity	Hard
Customer/Stakeholder	Department Course Faculty / Linux Systems Developers

1.2 Team Profile and Product Ownership

Member	SRN	Owned Functional Feature	Secondary / Qualitative Responsibility
Anshul Poovaiah K	PES1202402145	Lexical Parsing, Tokenization & Environment Engine — input line ingestion, whitespace/operator tokenization (`	, <, >, >>, 2>, &), single and double quotation parsing ('...', "..."), escape sequences, variable resolution (\$VAR, \$?), and environment built-in commands (export, unset, env`).
Archana Shivakumar	PES1202400638	Process Execution & Multi-Stage Pipeline Engine — binary path discovery via PATH, child process creation (fork), program execution (execvp/execve), multi-stage pipeline IPC chaining (`cmd1	cmd2
Ashlesh R V	PES1202403566	I/O Redirection & Asynchronous Job Control — file stream	CI/CD automation pipeline (GitHub Actions/Jenkins)

Member	SRN	Owned Functional Feature	Secondary / Qualitative Responsibility
		redirection (<, >, >>, 2>, &>), background command execution (&), POSIX signal handling (<code>sigaction</code> for <code>SIGINT</code> , <code>SIGTSTP</code> , <code>SIGCHLD</code>), parent shell insulation, and job control built-in utilities (<code>jobs</code> , <code>fg</code> , <code>bg</code>).	and Docker containerization.
Atishay Jain	PES1202400621	Shell Built-in Commands, History & Alias Subsystem — execution of parent-context built-ins (<code>cd</code> with <code>~</code> and <code>-</code> , <code>pwd</code> , <code>echo</code> with <code>-n</code> , <code>exit</code>), persistent command history with file serialization (<code>~/simple_shell_history</code>), alias table management (<code>alias</code> , <code>unalias</code>), and dynamic alias expansion.	Automated unit test framework (Unity), Valgrind memory leak verification, and Requirement Traceability Matrix (RTM).

1.3 Proposed Project Description

The Simple Shell is a lightweight, POSIX-compliant command-line interpreter developed in C for Unix/Linux operating environments. It acts as the primary user-space interface to the operating system kernel, reading user input through a Read-Eval-Print Loop (REPL), tokenizing command streams, resolving environment variables, orchestrating concurrent processes, managing inter-process communication pipelines, redirecting I/O streams, and handling

asynchronous signals and background jobs. The system provides developers and automated scripts with a dependable, resource-efficient execution environment built with strict modularity and zero memory leaks.

1.4 Problem Statement

Standard Linux command execution relies heavily on process isolation, accurate stream redirection, and low-latency IPC. Developing a custom command-line interpreter in C presents critical systems-level challenges: improper process management causes zombie accumulation or fork bombs; unclosed pipe file descriptors lead to pipeline deadlocks; faulty signal delegation terminates the parent shell prematurely; and dynamic string allocations in interactive loops often cause severe memory leaks. The Simple Shell project addresses these challenges through a modular, strictly audited POSIX implementation developed under Agile discipline, ensuring high throughput, deterministic child process management, and complete memory safety.

1.5 Target Users

User Type	Primary Need	Major Activities
Interactive Developer / Sysadmin	Execute operating system binaries, chain CLI tools, and manage long-running jobs.	Launch interactive shell prompt, execute commands, connect tools via pipelines, redirect file streams, manage background jobs, inspect history, and configure aliases.
Script / Automation Runner	Execute non-interactive sequential command scripts reliably with deterministic exit status codes.	Pass command streams via standard input or script files, run automated utility workflows, and check \$? exit status codes for control flow.

1.6 Objectives

- Provide a robust, interactive REPL interface with dynamic prompt rendering (username, hostname, current directory).
- Implement an advanced lexical tokenizer supporting nested whitespace, single quotes, double quotes, and escape sequences.
- Support dynamic environment variable expansion (`$VAR`) and the previous command exit status token (`$?`).
- Enable seamless binary resolution across the system `PATH` using POSIX `fork`, `execvp`, and `waitpid`.
- Implement arbitrary-depth multi-stage command pipelines (`cmd1 | cmd2 | ... | cmdN`) using Unix pipes and `dup2`.
- Provide comprehensive file stream redirection for stdin (`<`), stdout truncate (`>`), stdout append (`>>`), and stderr (`2>`).
- Implement asynchronous background processing (`&`) and job control (`jobs`, `fg`, `bg`) with non-blocking `SIGCHLD` reaping.
- Insulate the parent shell against interactive termination signals (`SIGINT`, `SIGTSTP`) while forwarding them to active foreground children.
- Implement parent-context built-ins (`cd`, `pwd`, `echo`, `export`, `unset`, `exit`) without child process spawning.
- Ensure persistent history tracking across sessions (`~/.simple_shell_history`) and zero memory leaks under Valgrind audits.

1.7 Scope

In Scope	Out of Scope
• Interactive REPL with custom prompt rendering and non-interactive batch mode.	• Graphical User Interface (GUI) or browser-based terminal.
• Lexical analysis with single/double quotation and escape character handling.	• Complex Bash scripting control structures (<code>if/then</code>, <code>for/while</code> loops, functions).

In Scope	Out of Scope
Environment variable resolution (<code>\$VAR</code>, <code>\$?</code>) and built-in env management (<code>export</code>, <code>unset</code>, <code>env</code>).	Automated network socket streaming or remote SSH daemon capabilities.
Binary path lookup via <code>PATH</code>, process execution (<code>fork/execvp</code>), and exit code harvesting.	Kernel-level device driver development or kernel module modifications.
Multi-stage unidirectional inter-process pipelines (<code>`</code>	<code>`</code>) of arbitrary depth.
File stream redirections (<code><</code>, <code>></code>, <code>>></code>, <code>2></code>, <code>&></code>).	Multi-terminal broadcast or multi-seat session sharing.
Background job management (<code>&</code>, <code>jobs</code>, <code>fg</code>, <code>bg</code>) and process table tracking.	Full POSIX terminal line-editing library implementation from scratch.
Asynchronous signal handling (<code>sigaction</code> for <code>SIGINT</code>, <code>SIGTSTP</code>, <code>SIGCHLD</code>).	High-level scripting runtime integrations (e.g., Python/Lua embedded interpreters).
Core parent built-ins (<code>cd</code>, <code>pwd</code>, <code>echo</code>, <code>exit</code>).	
Persistent session history file serialization (<code>~/.simple_shell_history</code>) and command aliases (<code>alias</code>, <code>unalias</code>).	

1.8 Functional Features

- Interactive Prompt & Line Ingestion:** Formatted prompt showing user, host, and working directory; robust line reading via `getline`.

2. **Lexical Tokenization & Syntax Validation:** Splits input into discrete tokens while honoring single/double quotation contexts and escape characters.
3. **Environment Variable Expansion:** Expands `$ENV` variables dynamically and evaluates `$?` to the exit status of the previous command.
4. **External Command Execution:** Scans `PATH`, creates child processes using `fork()`, replaces image with `execvp()`, and collects exit codes via `waitpid()`.
5. **Multi-Stage Pipeline IPC:** Chains commands using unidirectional Unix pipes (`pipe()`, `dup2()`) with strict closure of unused file descriptors to prevent deadlocks.
6. **File Stream Redirection:** Routes standard input from files (`<`), redirects standard output with truncate (`>`) or append (`>>`), and redirects standard error (`2>`).
7. **Background Job Execution:** Executes non-blocking commands suffixed with `&`, outputs job/PID notifications, and maintains responsive parent prompt.
8. **Asynchronous Signal Handling & Zombie Reaping:** Uses `sigaction` to isolate the parent from `SIGINT/SIGTSTP` and reaps background children cleanly via `SIGCHLD`.
9. **Parent-Context Built-in Commands:** Directly modifies parent shell process state for `cd`, `pwd`, `echo`, `export`, `unset`, and `exit`.
10. **Command History Serialization & Aliases:** Maintains an in-memory history ring buffer, serializes history to disk on exit, and expands user-defined command aliases.

1.9 Proposed Technology / Tool Stack

Area	Proposed Technology / Tool
Core Systems Language	C (C11 Standard / POSIX.1-2008)
Compiler & Toolchain	GCC (with <code>-Wall</code> <code>-Wextra</code> <code>-Werror</code> <code>-pedantic</code>)
Build Automation	GNU Make

Area	Proposed Technology / Tool
Operating System & Kernel	Ubuntu 22.04 LTS / Debian 12 (Linux Kernel 5.15+)
Version Control	Git + GitHub
Agile / Issue Tracking	Jira or GitHub Projects
Automated Unit Testing	Unity / Criterion (C Testing Frameworks)
Integration Testing	Bash Automated Test Harness (<code>tests/integration_tests.sh</code>)
Memory & Leak Sanitization	Valgrind (<code>--leak-check=full</code>) + AddressSanitizer (ASan)
Static Code Analysis	Cppcheck / Clang-Tidy
Containerization	Docker
CI/CD Automation	GitHub Actions / Jenkins

1.10 Agile Methodology Justification

The Simple Shell project will follow the Agile Scrum framework across six two-week sprints because operating systems software requires incremental development and continuous verification. Developing core components—lexical parsing, process execution, pipelines, redirection, job control, and built-ins—in isolation before integrating them reduces concurrency bugs and resource deadlocks. Frequent sprint reviews, automated CI regression testing, and continuous Valgrind memory leak audits allow the team to detect pointer bugs and descriptor leaks early, ensuring steady progress toward an industrial-grade, POSIX-compliant final executable.

1.11 Short-Term Plan

Iteration	Planned Output	Acceptance Evidence
Sprint 1	Project setup, Makefile/GCC configuration, REPL prompt loop, basic tokenizer, single external command execution (<code>fork/execvp/waitpid</code>).	User stories + Git repo + clean compilation with <code>-Wall -Wextra</code> + working single command execution.
Sprint 2	Advanced quoting support (single/double), environment variable expansion (<code>\$VAR</code> , <code>\$?</code>), and parent built-ins (<code>cd</code> , <code>pwd</code> , <code>exit</code> , <code>export</code>).	Automated unit test suite demonstrating quote handling + verified working directory state updates.
Sprint 3	File stream redirection (<code><</code> , <code>></code> , <code>>></code> , <code>2></code>) and multi-stage pipeline orchestration (<code>pipe</code> , <code>dup2</code>).	Integration test script running 2-pipe and 3-pipe commands; output parity verified against <code>/bin/sh</code> .
Sprint 4	Signal handling (<code>sigaction</code> for <code>SIGINT</code> , <code>SIGTSTP</code>), background execution (<code>&</code>), job table tracking, and <code>jobs/fg/bg</code> built-ins.	Interactive terminal tests confirming parent shell survival on <code>Ctrl+C/Ctrl+Z</code> ; zero zombie processes.
Sprint 5	History serialization (<code>~/.simple_shell_history</code>), alias engine, static analysis cleanup, and	Cppcheck scan showing 0 defects; Valgrind execution reporting "All heap blocks were freed -- no leaks are possible".

Iteration	Planned Output	Acceptance Evidence
	Valgrind memory leak fixes.	
Sprint 6	System regression test suite, Docker container build, CI/CD pipeline completion, and final demo dry run.	Passing end-to-end regression test suite, tagged GitHub v1.0 release, and finalized presentation slides.